

CAPSTONE PROJECT REPORT

AI-Powered Financial & Tax Analysis Tool

AICA Level 2 – Batch 83 (Surat)
Prepared by: CA Shivangi Mehta

Application identity: FINsight – Offline Financial Review & Compliance Assistant

1. Executive Summary

FINsight is an offline-first financial review and compliance application developed to structure the review of accounting data and to surface accounting, audit and tax exceptions for professional review. The submitted project combines a Windows packaged application, a Python source codebase, local SQLite storage, financial-data ingestion, validation and mapping, specialized review engines, a unified findings workflow, risk scoring, working-paper/query support, and security hardening.

2. Problem Statement

Accounting and audit professionals routinely review trial balances, general ledgers, purchase and sales registers, fixed-asset information and other structured exports. The work involves repeated validation, cross-checking, rule application and documentation. A structured software workflow can improve consistency, traceability and review efficiency while keeping the professional responsible for final judgement.

3. Project Objectives

The project objectives are to provide a repeatable local workflow for importing financial data, validating and mapping data structures, running accounting/audit/tax checks, organizing findings, supporting engagement-level review and producing structured outputs without requiring cloud connectivity.

4. Proposed Solution

FINsight provides a single application workflow in which financial files are uploaded, validated and mapped before review. Separate accounting, audit and tax engines apply their own rule sets. A unified review layer provides readiness gating, module orchestration and normalized findings for presentation.

5. Functional Workflow

The overall workflow is: **Engagement** → **Upload** → **Structure Detection / Mapping** → **Data Validation** → **Review Readiness** → **Accounting + Audit + Tax Review** → **Findings Centre** → **Risk / Queries / Reports**. The unified review stage prevents a full engagement review from running until all uploaded files required for that engagement have reached the required validation state.

6. Application Architecture

The source tree separates Flask blueprints/API routes, service-layer logic, rule modules, data models, mapping/validation components and frontend templates/static assets. SQLite is used for application data. Local input, processed and output directories are used for files. The packaged Windows entry point supports local mode and controlled private-LAN mode.

7. Accounting Review

The accounting rule layer contains 10 dedicated accounting rule modules in the submitted source tree, covering areas such as fixed assets, depreciation, inventories, provisions, borrowing costs, employee benefits, related-party/other accounting presentation and shared detectors.

8. Audit Review

The audit rule layer contains 25 dedicated audit rule modules. The source tree includes checks relating to ledgers, revenue, journal entries, cut-off, estimates, liabilities, cash, write-offs, related review areas and shared ledger scrutiny.

9. Tax Review

The tax rule layer contains 9 dedicated tax rule modules, including GST, cash transactions, depreciation, loans, MSME, tax audit, disallowance and related tax-review areas. The repository also contains a documented tax-rule source-verification approach.

10. Unified Review Engine

The unified review service orchestrates accounting, audit and tax review modules, applies a review-readiness gate, normalizes findings and supplies a common Findings Centre. Importantly, the architecture documentation records that the orchestration layer reuses the underlying review engines rather than duplicating their rule logic.

11. Data and Database Layer

SQLite is used as the local database. SQLAlchemy provides the ORM/data-access layer. Uploaded files are stored under engagement-specific local folders. The design uses a local database and filesystem rather than a mandatory cloud service, supporting the offline-first objective.

12. Security and Privacy

The supplied security documentation records a source-level review of network behaviour, external API usage, upload handling, path traversal, engagement isolation, IDOR, SQL injection, session handling, error handling and filesystem access. The application is designed to keep client data on the host machine and to avoid outbound network calls in its own application source.

13. Offline and AI Position

The submitted source documentation states that no external AI provider is wired into the live application and that the AI feature flag is disabled by default. Therefore, the current demonstrable core is an offline, rule-driven financial review/compliance engine. This distinction should be stated transparently during presentation even though the selected capstone title is “AI-Powered Financial & Tax Analysis Tool.”

14. Technology Stack

The project uses Python with Flask, SQLAlchemy, SQLite, Pandas, OpenPyXL, ReportLab, Pydantic, Waitress and Pytest/pytest-cov. The Windows packaging configuration uses PyInstaller. The application includes approximately 188 Python source files in the submitted tree after cache cleanup.

15. Testing Strategy

The repository includes a broad automated test suite covering application factory behaviour, accounting, audit, tax, upload/validation, mapping, unified review, dashboard, engagement, queries, security, LAN mode and packaging. The submitted source tree contains 44 named test files. A human-readable test matrix is supplied separately with this package.

16. Sample Financial Data

The uploaded project contains 5 current sample financial input workbooks: Trial Balance, General Ledgers, Purchase Register, Sales Register and Fixed Asset Register. These have been copied into the Capstone package for convenient demonstration. Additional separate demonstration sets can be added if they are maintained outside the uploaded ZIP.

17. Expected Professional Benefits

The application can reduce repetitive screening work, provide a consistent rule-driven first-pass review, organize exceptions into a common findings workflow, and provide a repeatable evidence trail for demonstration and review. It is intended as an assistant to professional review rather than a replacement for professional judgement.

18. Limitations

Results depend on source data quality and the applicability of the selected rules. A flagged exception is not automatically a confirmed error; it is an item requiring professional review. The current live AI provider integration is disabled. Windows EXE behaviour should be verified on the final target computer before production use.

19. Future Scope

Potential extensions include optional AI-assisted explanations, expanded GST/direct-tax coverage, richer reconciliation analytics, more automated report generation, broader source formats, enhanced dashboards and additional engagement-level analytics.

20. Conclusion

FINsight demonstrates an end-to-end software approach to financial, accounting, audit and tax review: data ingestion, validation, structured rule execution, unified findings, risk-oriented review and professional documentation, all within an offline-first architecture. The project combines practical CA-domain requirements with a modular Python application design.

21. High-Level Architecture Diagram

User / Reviewer
FINsight Web UI
Flask API / Blueprints
Services: Upload • Mapping • Validation • Review • Risk • Queries • Reports
Accounting Rules Audit Rules Tax Rules
SQLite Database + Local File Storage
Findings • Risk • Working Papers • Reports

The architecture keeps review logic in dedicated services/rules and keeps the UI layer separate from direct database operations.